

## Optimisation combinatoire : application

Participants : Domo Adama, Pelletier Hubert, Sako Merouane, Thuau Sébastien

### Rappel du problème :

Notre but est de résoudre un problème d'organisation de soirée. On dispose d'un ensemble de  $N$  convives potentiels. Pour chaque convive potentiel  $i \in \{1, \dots, N\}$ , on définit  $V_i$  l'ensemble de ses connaissances, avec  $V_i \subseteq \{1, \dots, N\}$ . Pour tout couple  $(i, j) \in \{1, \dots, N\}^2$ , si  $i$  et  $j$  se connaissent, alors  $i \in V_j$  et  $j \in V_i$ . Tout convive potentiel  $i$  a une valeur  $c_i$  représentant l'intérêt d'avoir cette personne à notre soirée. Pour une instance donnée, on cherchera à réunir le plus de personnes intéressantes possibles à condition que toutes se connaissent : l'objectif sera donc dépendant du coefficient d'intérêt  $c_i$  de chacun des convives invités.

### Modèle linéaire :

Soit  $x_i$  une variable de décision tel que :

$x_i : \begin{cases} 1 & \text{si } i \text{ est invité} \\ 0 & \text{sinon} \end{cases}$

### **Objectif**

Maximiser  $\sum_{i=1}^N c_i \times x_i$

### **S.C :**

- Si  $i$  est invité et ne connaît pas  $j$  alors  $j$  ne doit pas être invité :

$x_i + x_j \leq 1$  pour tout  $i, j$  tel que  $i \notin V_j$  et  $j \notin V_i$

### Résultats trouvés avec le solveur GLPK

| Instances   | Score trouvé |
|-------------|--------------|
| Instance 1  | 73           |
| Instance 2  | 91           |
| Instance 3  | 84           |
| Instance 4  | 83           |
| Instance 5  | 81           |
| Instance 6  | 80           |
| Instance 7  | 81           |
| Instance 8  | 85           |
| Instance 9  | 77           |
| Instance 10 | 91           |

### **Résultats trouvés par l'algorithme glouton**

De manière générale, les tests ont été réalisés sur Windows. Pour l'algorithme glouton, nous avons codé le même algorithme glouton que dans le sujet. Nous retrouvons correctement les mêmes résultats.

| Instances   | Score | Gap    |
|-------------|-------|--------|
| Instance 1  | 69    | 0.05   |
| Instance 2  | 90    | 0.01   |
| Instance 3  | 71    | 0.15   |
| Instance 4  | 67    | 0.19   |
| Instance 5  | 80    | 0.01   |
| Instance 6  | 73    | 0.0875 |
| Instance 7  | 61    | 0.25   |
| Instance 8  | 69    | 0.19   |
| Instance 9  | 68    | 0.12   |
| Instance 10 | 72    | 0.21   |

Pour le glouton randomisé, nous masquons aléatoirement une valeur sur deux du jeu de données afin de créer de la diversité.

Pour le glouton qui sert à réparer les solutions, nous masquons toutes les valeurs qui ne sont pas dans l'ensemble de connaissances des individus de la solution passée en entrée.

### **Résultats trouvés par l'algorithme génétique**

Les résultats sont sur 5 essais par instance avec une taille de population de 400 et une taille de sélection de 100. La sélection se fait de manière déterministe en prenant les 400 meilleurs individus. Pour plus de simplicité, le croisement est à 2 points. La probabilité de croisement est de 80%. Comme dans le sujet, la probabilité de mutation est de  $2/N$ . L'entraînement se fait en cinq générations par essai. Plus tard, nous avons testé avec 60 secondes d'exécution et nous trouvions souvent la solution optimale.

Nombre de tours de boucles par essai = 5

Petites instances

| Instances   | Score min | Score moyen | Score max |
|-------------|-----------|-------------|-----------|
| Instance 1  | 71        | 71.4        | 73        |
| Instance 2  | 91        | 91          | 91        |
| Instance 3  | 84        | 84          | 84        |
| Instance 4  | 81        | 82.6        | 83        |
| Instance 5  | 81        | 81          | 81        |
| Instance 6  | 76        | 77.6        | 80        |
| Instance 7  | 76        | 78.4        | 81        |
| Instance 8  | 83        | 84.6        | 85        |
| Instance 9  | 75        | 75.4        | 76        |
| Instance 10 | 87        | 90.2        | 91        |

Pour les grandes instances, une seule génération a été réalisé à cause du temps d'exécution.

#### Grandes instances

| Instances   | Score min | Score moyen | Score max |
|-------------|-----------|-------------|-----------|
| Instance 1  | 567       | 570.4       | 575       |
| Instance 2  | 572       | 580.8       | 596       |
| Instance 3  | 608       | 609         | 611       |
| Instance 4  | 581       | 589.6       | 604       |
| Instance 5  | 555       | 566         | 573       |
| Instance 6  | 562       | 564.8       | 568       |
| Instance 7  | 557       | 563         | 574       |
| Instance 8  | 559       | 563         | 568       |
| Instance 9  | 541       | 544         | 549       |
| Instance 10 | 619       | 622.6       | 629       |

Afin d'avoir plus de générations, nous avons réduit la taille de la population de 120 et la taille de sélection à 60. Les résultats suivants sont sur Linux pour un temps d'exécution de 120 secondes.

| Instances   | Score trouvé |
|-------------|--------------|
| Instance 1  | 604          |
| Instance 2  | 620          |
| Instance 3  | 643          |
| Instance 4  | 620          |
| Instance 5  | 593          |
| Instance 6  | 598          |
| Instance 7  | 590          |
| Instance 8  | 595          |
| Instance 9  | 577          |
| Instance 10 | 658          |

Il y a une nette amélioration de la précision dû à l'augmentation du nombre de générations et il semble que nos résultats sont meilleurs sur Linux que sur Windows pour les grandes instances.

### Résultats de l'algorithme à liste tabou

Nous avons codé un algorithme à liste tabou testant le voisinage en enlevant deux individus de la solution puis en appliquant un glouton qui répare sur cette nouvelle solution considérant les valeurs enlevées comme des valeurs interdites pour le glouton. De nouvelles solutions sont cherchées de manière récurrente jusqu'à ce qu'il n'y ait plus de voisin qui améliore le score. Les résultats du tableau ci-dessous sont basés d'une initialisation de l'algorithme à partir de la solution trouvée par notre algorithme glouton.

| Petites instances | Score |
|-------------------|-------|
| Instance 1        | 70    |
| Instance 2        | 90    |
| Instance 3        | 74    |
| Instance 4        | 68    |
| Instance 5        | 80    |
| Instance 6        | 73    |
| Instance 7        | 73    |
| Instance 8        | 73    |
| Instance 9        | 71    |
| Instance 10       | 88    |

La récurrence prenant beaucoup de temps, nous avons par la suite limiter la recherche de voisinage en enlevant seulement un seul voisin et en limitant la récurrence à une profondeur de 4. Les résultats semblent meilleurs que l'algorithme glouton tout en étant moins bon que l'algorithme génétique. Nous avons aussi testé un fonctionnement itératif de cet algorithme pour qu'il fonctionne sur des instances larges mais les résultats n'étaient pas concluant tout en rajoutant inutilement du temps de calcul. Cependant, l'algorithme à liste tabou contribue à améliorer les solutions ce qui nous a mené à l'utiliser après la réparation des enfants afin d'améliorer ceux-ci.

### **Résultats de l'algorithme génétique avec hybridation**

Pour avoir un temps d'exécution raisonnable, nous avons défini une profondeur maximum de récursivité de 4 et nous n'enlevons plus qu'une seule valeur à la fois pour tester le voisinage. Malgré cela, nous n'avons qu'un temps raisonnable que sur les petites instances.

Pour ces résultats, la probabilité qu'on utilise l'algorithme à liste tabou sur l'enfant est de 75%.

Pour le reste, les conditions de test sont les mêmes que pour l'algo génétique simple.

| Instances   | Score min | Score moyen | Score max |
|-------------|-----------|-------------|-----------|
| Instance 1  | 73        | 73          | 73        |
| Instance 2  | 91        | 91          | 91        |
| Instance 3  | 84        | 84          | 84        |
| Instance 4  | 83        | 83          | 83        |
| Instance 5  | 81        | 81          | 81        |
| Instance 6  | 80        | 80          | 80        |
| Instance 7  | 81        | 81          | 81        |
| Instance 8  | 85        | 85          | 85        |
| Instance 9  | 77        | 77          | 77        |
| Instance 10 | 91        | 91          | 91        |

Nous trouvons quasiment tout le temps la solution optimale pour cet algorithme, l'algorithme à liste tabou permet une convergence plus rapide vers une solution.